Aula 18 — Compartilhando regras

> **Sem perder os contratos**

Quando classes diferentes podem compartilhar implementação sem perder suas responsabilidades?

IA — Nível 1: Tutor. Faça as previsões antes de consultar ajuda.

DA AULA 17 — O QUE JÁ FUNCIONA

A bigorna conhece a capacidade de reparo

Item	Durabilidade	Material	Reparo
Espada de ferro	`40/100`	ferro	`+25` , até `100`
Escudo	`30/80`	madeira	`+20` , até `80`
Maçã	não possui	nenhum	não oferece

O inventário usa **Item** para apresentar. A bigorna usa **Reparavel** para solicitar o reparo.

ATIVIDADE

RECUPERAÇÃO — CENÁRIO INDEPENDENTE

Quem muda cada estado?

```
Escudo escudo = new Escudo(30);
Reparavel equipamento = escudo;
PilhaMaterial madeira = new PilhaMaterial("madeira", 2);
boolean reparou = equipamento.repararCom(madeira);
```

1. Qual é o retorno, a durabilidade final e o saldo?
2. Quem altera a durabilidade? Quem altera a quantidade?

Dois objetos cuidam de estados diferentes

Observação	Resultado
Retorno	`true`
Durabilidade	`50/80`
Madeira restante	`1`

O **escudo** altera sua durabilidade; a **pilha** altera a quantidade quando recebe `consumirUma()`.

`equipamento` e `escudo` alcançam o mesmo escudo.

O reparo confere, consome e respeita o limite

```
if (!material.ehDoTipo("ferro") || durabilidade == 100) {
    return false;
}
if (!material.consumirUma()) {
    return false;
}
durabilidade += 25;
if (durabilidade > 100) {
    durabilidade = 100;
}
return true;
```

ATIVIDADE

AGORA COMPARE — CORPO DE REPARARCOM EM ESCUDO

O que varia e o que se repete?

```
if (!material.ehDoTipo("madeira") || durabilidade == 80) {
    return false;
}
if (!material.consumirUma()) {
    return false;
}
durabilidade += 20;
if (durabilidade > 80) {
    durabilidade = 80;
}
return true;
```

Quais três dados variam? O que se repete? Onde corrigir o limite?

A REPETIÇÃO ESTÁ NO ALGORITMO, NÃO SÓ NOS NOMES DOS CAMPOS

Uma sequência; três parâmetros diferentes

Dado	Espada	Escudo
Material aceito	`"ferro"`	`"madeira"`
Máximo	`100`	`80`
Acréscimo	`25`	`20`

Conferir → consumir → aumentar → limitar → informar sucesso.

Hoje precisamos conferir uma correção nos **dois métodos**. Podemos concentrar essa regra e conservar os dados de cada equipamento.

COMPARTILHAR O REPARO CRIA UMA PERGUNTA SOBRE A DESCRIÇÃO

A base conhece as regras; cada item se apresenta

Espada e escudo são equipamentos reparáveis neste modelo.

Uma base **Equipamento** pode concentrar o estado e o reparo. Mas qual seria a descrição de um equipamento genérico?

A base será **abstrata**: deixa `descrever()` pendente para cada classe concreta.

EQUIPAMENTO.JAVA — DECLARAÇÃO E CAMPOS

A implementação comum conserva os dois contratos

```
public abstract class Equipamento implements Item, Reparavel {
    private int durabilidade;
    private int maximo;
    private String materialAceito;
    private int acrescimo;
    // construtor e métodos nos próximos recortes
}
```

Os campos são privados. Cada equipamento concreto terá seus próprios valores.

Os parâmetros recebem os dados de cada equipamento

```
protected Equipamento(int durabilidadeInicial, int maximo,
                      String materialAceito, int acrescimo) {
    this.durabilidade = durabilidadeInicial;
    this.maximo = maximo;
    this.materialAceito = materialAceito;
    this.acrescimo = acrescimo;
}
```

Por enquanto, os argumentos são válidos. O construtor apenas inicializa campos.

A consulta tem corpo; a descrição fica pendente

```
@Override
public int getDurabilidade() {
    return durabilidade;
}

@Override
public abstract String descrever();
```

`descrever()` termina com ponto e vírgula. **Não há corpo** para executar na base.

ESPADA.JAVA — SUBSTITUI A CLASSE ANTERIOR

A espada informa os dados e implementa a descrição

```
public class Espada extends Equipamento {
    public Espada(int durabilidadeInicial) {
        super(durabilidadeInicial, 100, "ferro", 25);
    }
    @Override
    public String descrever() {
        return "Espada de ferro: " + getDurabilidade() + "/100";
    }
}
```

Não conservamos outra cópia da durabilidade nem do método de reparo.

O escudo conserva sua descrição e seus limites

```
public class Escudo extends Equipamento {
    public Escudo(int durabilidadeInicial) {
        super(durabilidadeInicial, 80, "madeira", 20);
    }
    @Override
    public String descrever() {
        return "Escudo: " + getDurabilidade() + "/80";
    }
}
```

A ordem dos argumentos é: inicial, máximo, material, acréscimo.

extends e super têm funções diferentes

- `extends Equipamento`: estabelece a superclasse.
- `super(...)`: chama o construtor dessa base.
- `protected` no construtor permite a chamada pelas subclasses.
- A espada também cumpre `Item` e `Reparavel`, sem repetir `implements`.

Os campos da base continuam **private**.

ATIVIDADE

CONSTRUÇÃO — CENÁRIO NOVO

Dois construtores criam dois objetos?

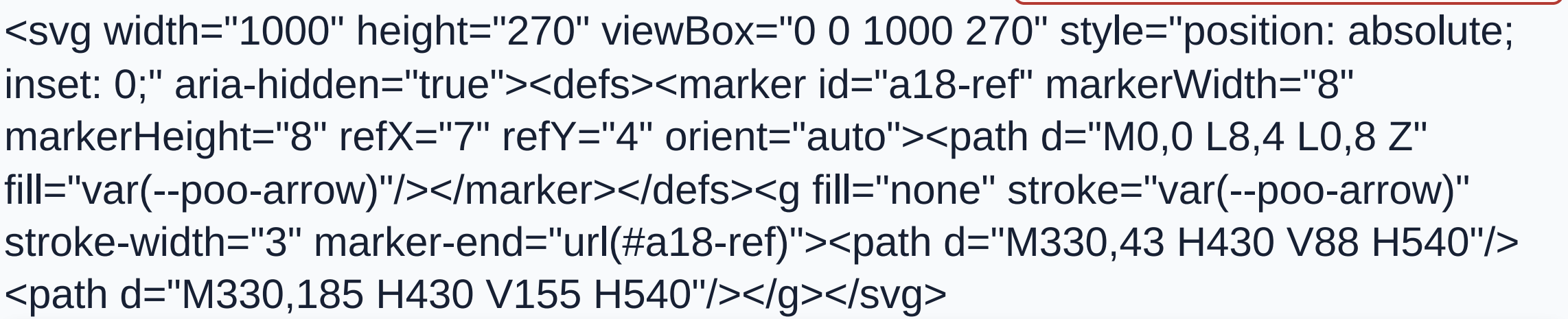
```
Espada espada = new Espada(40);
Equipamento equipamento = espada;
```

1. Quantos equipamentos são criados? As referências alcançam quais objetos?
2. Que construtor inicializa os campos de durabilidade e reparo?

As duas referências alcançam a mesma espada

espada : Espada

equipamento : Equipamento



RECONSTRUIR A CRIAÇÃO — NEW ESPADA(40)

A base participa da inicialização da espada

1. `Espada` recebe `40` em seu construtor.
2. `super(...)` passa `40`, `100`, `"ferro"` e `25` à base.
3. `Equipamento` atribui os valores aos campos do mesmo objeto.
4. A construção termina; a referência é atribuída a `espada`.

Construtores **não são herdados**. A espada declara o seu e chama o da base.

ATIVIDADE

DENTRO DE ESPADA — DUAS PROPOSTAS DE DESCRIÇÃO

Herdar permite acessar qualquer campo?

A

```
return "Espada de ferro: " + durabilidade + "/100";
```

B

```
return "Espada de ferro: " + getDurabilidade() + "/100";
```

Qual compila? Precisamos tornar o campo `protected`?

A VISIBILIDADE CONTINUA SENDO RESPEITADA

B usa a consulta pública herdada

B compila. `getDurabilidade()` está implementado em `Equipamento` e é público.

A não compila. `durabilidade` é privado em outra classe.

Não precisamos ampliar a visibilidade do campo. A base continua responsável por proteger e alterar esse estado.

ATIVIDADE

DIAGNÓSTICO — ANALISAR CADA LINHA SEPARADAMENTE

A base abstrata pode ser um tipo de referência?

```
Equipamento a = new Espada(40);
Equipamento b = new Equipamento(40, 100, "ferro", 25);
Item c = new Escudo(30);
Reparavel d = new Espada(40);
```

Quais linhas compilam? Por que declarar uma referência é diferente de instanciar a classe?

REFERENCIAR UM OBJETO NÃO EXIGE INSTANCIAR O TIPO DECLARADO

a, c e d compilam; b tenta criar a base abstrata

Linha	Diagnóstico
`a`	uma espada pode ser usada como `Equipamento`
`b`	`abstract` impede `new Equipamento(...)`
`c`	um escudo cumpre `Item`
`d`	uma espada cumpre `Reparavel`

Uma subclasse concreta precisa fornecer as operações abstratas pendentes. Caso contrário, ela também deverá ser abstrata.

As recusas usam os dados do próprio objeto

```
@Override
public boolean repararCom(PilhaMaterial material) {
    if (!material.ehDoTipo(materialAceito) || durabilidade == maximo) {
        return false;
    }
    if (!material.consumirUma()) {
        return false;
    }
    // continuação no próximo slide
}
```

A mesma regra protege os limites de cada equipamento

```
durabilidade += acrescimo;
if (durabilidade > maximo) {
    durabilidade = maximo;
}
return true;
```

Na espada: `+25`, até `100`. No escudo: `+20`, até `80`.

O corpo está na base; o estado pertence ao equipamento que recebe a chamada.

CONCEITO-CHAVE

A HIERARQUIA JÁ TEM ESTADO E COMPORTAMENTO

Herdar e sobrescrever são papéis diferentes

Reparo: a espada usa a implementação herdada de `Equipamento`.

Descrição: a espada fornece uma implementação para a operação abstrata. Isso é **sobrescrita**.

Ambas executam sobre o próprio objeto concreto.

ATIVIDADE

CENÁRIO NOVO — QUATRO REFERÊNCIAS PARA UMA ESPADA

Quais implementações executam?

```
Espada espada = new Espada(40);
Item item = espada;
Equipamento equipamento = espada;
Reparavel capacidade = espada;
PilhaMaterial ferro = new PilhaMaterial("ferro", 2);
System.out.println(item.descrever());
System.out.println(capacidade.repararCom(ferro));
System.out.println(equipamento.descrever());
System.out.println(espada.getDurabilidade());
System.out.println(ferro.getQuantidade());
System.out.println(item == equipamento);
```

Preveja as seis linhas. Onde estão os corpos do reparo e da descrição?

AS REFERÊNCIAS OBSERVAM O MESMO ESTADO REPARADO

Uma espada, duas descrições em momentos diferentes

```
Espada de ferro: 40/100
true
Espada de ferro: 65/100
65
1
true
```

O reparo executa o corpo de **Equipamento**. A descrição executa o de **Espada**. Há um único equipamento, alcançado pelas quatro referências.

Declarar a operação permite solicitar sua execução

Chamada	Compila?	Motivo
`capacidade.descrever()`	não	`Reparavel` não declara a operação
`equipamento.descrever()`	sim	`Equipamento` declara a operação abstrata

`@Override` ajuda o compilador a detectar um nome errado, como `descrver()`.

A anotação não escolhe o corpo na execução: isso continua dependendo do objeto.

A bigorna ainda solicita uma capacidade

```
public boolean tentarReparar(Item item, PilhaMaterial material) {
    if (item instanceof Reparavel) {
        Reparavel equipamento = (Reparavel) item;
        return equipamento.repararCom(material);
    }
    return false;
}
```

`Inventario` conserva sua `List<Item>`. `Maca` continua implementando somente `Item`.

ATIVIDADE

A NOVA BASE DEVE MUDAR TODOS OS CLIENTES?

Compare três propostas

A. A bigorna testa `instanceof Equipamento`, pois os reparáveis atuais usam essa base.

B. A maçã estende `Equipamento`, para todos os itens ficarem na mesma hierarquia.

C. A bigorna usa `Reparavel`; a base fica onde suas regras fazem sentido.

Qual preserva os papéis? Um futuro `Item` reparável de outra hierarquia seria aceito pela bigorna atual?

C preserva os contratos dos clientes

- **A** restringe a bigorna à família `Equipamento`.
- **B** atribui à maçã estado e reparo que não pertencem a seu papel.
- **C** mantém a colaboração pela capacidade necessária.

Um futuro objeto que cumpra **Item e Reparavel** pode usar a bigorna atual sem estender `Equipamento`.

A entrega com seguro guarda uma entrega-base

Recortes de `EntregaComSeguro`:

```
private Entrega entregaBase; // recebida no construtor

public double calcularCusto(Pedido pedido) {
    return entregaBase.calcularCusto(pedido) + 5.0;
}
```

Dois objetos colaboram. Já `Espada extends Equipamento` usa métodos que operam sobre o estado da própria espada.

ATIVIDADE

DUAS NECESSIDADES INDEPENDENTES

Escolher pelo requisito

1. Uma ferramenta segue a mesma regra de reparo, mudando máximo, material, acréscimo e descrição. Que solução pode compartilhar o algoritmo? Sob qual condição?
2. O seguro deve combinar com modalidades diferentes, recebidas na criação. O que se perde criando uma subclasse com seguro para cada modalidade?

OS REQUISITOS INDICAM RELAÇÕES DIFERENTES

Compartilhar regras ou combinar colaboradores

1. Uma subclasse de **Equipamento** pode passar seus dados à base e implementar a descrição, desde que respeite suas operações e regras.
2. Uma classe com seguro por modalidade repete o acréscimo e exige novas classes para novas modalidades. A **composição** permite receber qualquer `Entrega` compatível e delegar o cálculo.

Em Java, uma classe tem uma superclasse direta e pode cumprir várias interfaces.

ARMADILHA

UMA SEMELHANÇA PODE SUGERIR A PERGUNTA ERRADA

Herdar só para aproveitar código

Código parecido é um indício para investigar.

Antes de criar a hierarquia, confira se as subclasses podem respeitar as operações e regras da base.

Se precisarem negar essas regras, reavalie a relação e a distribuição das responsabilidades.

ATIVIDADE

DEPOIS DA EXTRAÇÃO — CENÁRIOS INDEPENDENTES

A organização mudou; os resultados mudaram?

Cenário	Estado inicial	Pilha
A	espada `90/100`	ferro `2`
B	escudo `30/80`	ferro `2`
C	escudo `30/80`	madeira `2`

Preveja o retorno, a durabilidade final e o saldo em cada cenário.

Refatorar inclui conferir o que deve permanecer

Cenário	Retorno	Durabilidade	Saldo
A	`true`	`100`	`1`
B	`false`	`30`	`2`
C	`true`	`50`	`1`

A: o limite impede `115`. **B:** material incompatível impede o consumo. **C:** o escudo usa acréscimo `20`.

Os testes anteriores continuam relevantes: construções e operações públicas foram preservadas.

Observar o limite, o consumo e a descrição

```
Espada espada = new Espada(90);
PilhaMaterial ferro = new PilhaMaterial("ferro", 2);
Bigorna bigorna = new Bigorna();

boolean reparou = bigorna.tentarReparar(espada, ferro);

assertEquals(true, reparou);
assertEquals(100, espada.getDurabilidade());
assertEquals(1, ferro.getQuantidade());
assertEquals("Espada de ferro: 100/100", espada.descrever());
```

UM TESTE NÃO REPRESENTA TODAS AS CONDIÇÕES DA REGRA

Preservar também os casos de recusa

O teste anterior cobre **limite, consumo e descrição** daquele cenário.

Precisamos conferir também:

- material incompatível, pilha vazia e equipamento completo;
- recusa da maçã, que não oferece reparo;
- dados específicos do escudo passados à base.

ATIVIDADE

FECHAMENTO INDIVIDUAL 1 — CENÁRIO INDEPENDENTE

A referência é Equipamento; o objeto é Escudo

```
Equipamento equipamento = new Escudo(30);
PilhaMaterial madeira = new PilhaMaterial("madeira", 2);
boolean reparou = equipamento.repararCom(madeira);
System.out.println(equipamento.descrever());
```

Qual corpo realiza o reparo? Qual é o retorno e a linha impressa? Quanto material resta?

FECHAMENTO 1 — O OBJETO CONSERVA OS DADOS DO ESCUDO

O reparo herdado altera a descrição observada

Executa **Equipamento.repararCom(...)** sobre o escudo.

Retorno: **true**. Madeira restante: **1**.

```
Escudo: 50/80
```

A linha é produzida por **Escudo.descrever()**, lendo a consulta herdada da durabilidade.

ATIVIDADE

FECHAMENTO INDIVIDUAL 2 — PROPOSTA ALTERNATIVA

E se o reparo voltasse para a bigorna?

Uma proposta apaga `reparar Com(...)` da base e coloca na bigorna um ramo para espada (`+25`) e outro para escudo (`+20`).

Mantendo os demais trechos:

1. As classes concretas ainda compilam? Por quê?
2. Que responsabilidade muda de lugar? O que acontece quando surge outro reparável?

FECHAMENTO 2 — MOVER UM CÁLCULO NÃO IMPLEMENTA O CONTRATO

A proposta fica incompleta e concentra regras no cliente

1. **Não compilam:** espada e escudo continuam cumprindo `Reparavel`, mas perderam a implementação de `repararCom(...)`.
2. A bigorna tenta assumir as regras dos equipamentos. Mesmo corrigindo a ausência do método, outra classe exigiria revisar seus ramos e dados.

Na solução atual, a bigorna coordena a tentativa; cada equipamento realiza o reparo.

ATIVIDADE

FECHAMENTO INDIVIDUAL 3 — INTEGRAR UMA NOVA FERRAMENTA

A picareta segue a mesma regra de reparo

Ferro; máximo **100**; acréscimo **25**. Descrição: `"Picareta: "` seguida da durabilidade e de `"/100"`.

1. Que partes precisam ser acrescentadas?
2. Proponha um cenário que verifique a integração sem alterar inventário ou bigorna: estado inicial, ação e resultados.

O reparo e os clientes existentes podem permanecer

Classe: `Picareta extends Equipamento`; construtor com `super(durabilidadeInicial, 100, "ferro", 25)`; sobrescrita de `descrever()`.

Montagem: criar a picareta e adicioná-la ao inventário.

Teste possível: picareta `90/100` no inventário, ferro `2`; reparar o item obtido. Esperar `true`, a mesma picareta, descrição `"Picareta: 100/100"` e saldo `1`.

PONTE PARA O LABORATÓRIO — MAIS UMA CONSEQUÊNCIA DO DESPACHO

Um método da base pode chamar uma operação sobrescrita

Imagine acrescentar a `Equipamento`:

```
public void apresentar() {
    System.out.println(descrever());
}
```

`descrever()` continua abstrato na base e implementado em cada subclasse concreta.

ATIVIDADE

CENÁRIO NOVO — APÓS ACRESCENTAR APRESENTAR() À BASE

Uma chamada leva a qual implementação?

```
new Espada(40).apresentar();
```

1. Que linha será impressa?
2. Onde estão os corpos de `apresentar()` e `descrever()` executados? As operações atuam sobre o mesmo objeto?

A CHAMADA INTERNA TAMBÉM USA O OBJETO CONCRETO

A base apresenta; a espada fornece a descrição

```
Espada de ferro: 40/100
```

1. Essa é a linha impressa.
2. `apresentar()` está em **Equipamento**; `descrever()` executa o corpo de **Espada**. Ambos atuam sobre a mesma espada.

A declaração abstrata permite a chamada. O objeto concreto fornece o corpo.

Os canais compartilham impressão e contagem

Uma base pode executar **obter identificação → imprimir → contar**. Cada canal fornece sua identificação: `SMS`, `PAINEL`, `APP` ou `EMAIL [Agenda]`.

Cada objeto conserva seu contador. O contador de **Lembrete** continua registrando os avisos daquele lembrete, que podem usar canais diferentes.

O objeto com prefixo continua delegando por **Notificador**, sem precisar pertencer à base dos canais de console.

SÍNTESE

COMPARTILHAR SEM PRENDER OS CLIENTES À BASE

Regras comuns, objetos próprios, contratos preservados

- A base concentra o estado e o reparo; cada equipamento tem seu estado.
- As subclasses herdam o reparo e sobrescrevem a descrição.
- A bigorna usa `Reparavel`: capacidade não exige uma mesma base.
- Herança e composição atendem relações e variações diferentes.

ATIVIDADE

APROFUNDAMENTO OPCIONAL — OUTRA DISTRIBUIÇÃO DO ESTADO

E se a durabilidade ficasse em um colaborador?

Espada e escudo implementam `Item` e `Reparavel` diretamente. Cada um cria seu próprio colaborador exclusivo para guardar a durabilidade e realizar o reparo.

1. Onde fica o estado? Que delegações são necessárias?
2. Qual vínculo deixa de ser exigido? Os contratos dos clientes precisam mudar?

A composição muda onde o estado vive

1. Durabilidade, limite e reparo ficam no **colaborador exclusivo**. Cada equipamento delega consulta e reparo; continua produzindo sua descrição.
2. Espada e escudo deixam de precisar estender `Equipamento`. Continuam cumprindo **Item e Reparavel**, preservando os clientes.

O algoritmo pode permanecer centralizado, com o custo das delegações e da construção do colaborador correto.